ST2195 Coursework

Programming Coursework



UNIVERSITY OF LONDON

Presented to: University of London, SIM Global Education

Course : Programming for Data Science (ST2195)

Prepared by: Darren Alexander

Lecturer: Chee Ming Leong

3 April 2025

Table of Contents

Part 1 2
Results of Part 1 (R code) 2
Explanations of Part 1 output (R code) 3
Results of Part 1 (Python code) 3
Explanations of Part 1 (Python code) 3

Part 1

The Metropolis-Hastings algorithm is a Markov Chain Monte Carlo (MCMC) method used for obtaining a sequence of random samples from a probability distribution for which direct sampling is difficult. Subsequently, there will be a calculation for the Gelman-Rubin convergence diagnostics ($\hat{R}$) for assessing convergence of multiple chains generated with different step size (s_values). In this report we will use R and Python code that will generate the sample distributions using visualisations such as histograms, kernel density plots, and provides an analysis of the results obtained from running the Metropolis-Hastings algorithm and assesses the convergence of the chain.

Task of Part 1a:

1. Simulate samples using the Random Walk Metropolis version of the Metropolis-Hastings algorithm.
2. Plot a histogram with a kernel density estimate and the true density overlay.
3. Compute the sample mean and sample standard deviation of the generated samples.

Task of Part 1b:

1. Calculate the Gelman-Rubin diagnostic ($\hat{R}$) for the Metropolis-Hastings Algorithm
2. Generate a plot of convergence using the Gelman-Rubin diagnostic ($\hat{R}$) over a range of s values.

Results of Part 1 (R code)

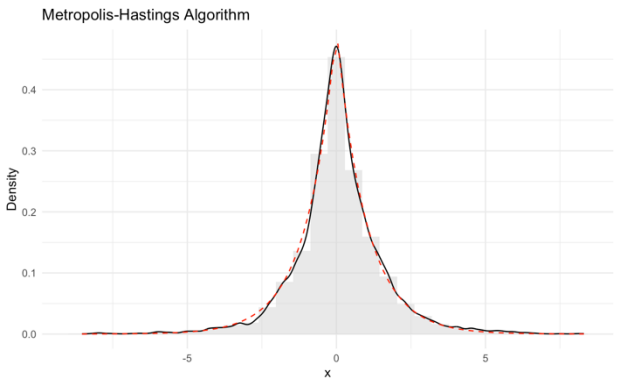


Figure 1.1: Part 1a R

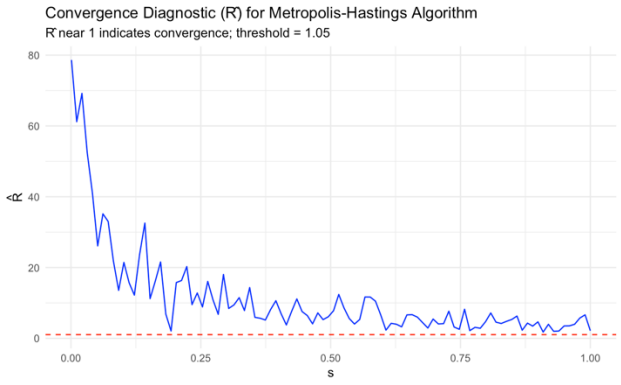


Figure 1.2: Part 1b R

Explanations of Part 1 output (R code)

1. *Figure 1.1* shows the histogram output, as well as the kernel density estimate and a true density overlay. Additionally, the generated sample mean and standard deviation is approximately 0.0613 and 1.5059 respectively. There is also an acceptance rate of 0.6975 which indicates that output values are reasonable.
2. *Figure 1.2* shows the convergence diagnostic for Metropolis-Hastings Algorithm over s values between 0.001 and 1. In the graph, high $\hat{R}$ is shown with small value of s, indicating poor convergence. Ultimately, as s values getting closer to 1, $\hat{R}$ will also get closer to 1, indicating convergence.

Results of Part 1 (Python code)

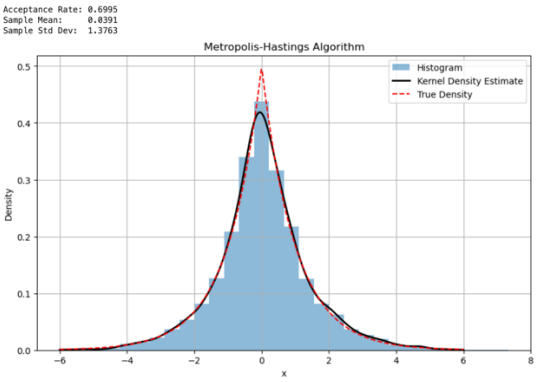


Figure 1.4: Part 1a Python

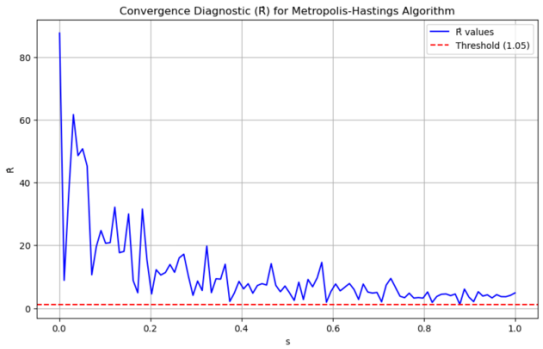


Figure 1.5: Part 1b Python

Explanations of Part 1 (Python code)

1. *Figure 1.4* shows the histogram overlaid with the kernel density estimate (KDE), as well as the true density. The printed statistics of the python code indicate that the sample mean is roughly 0.0391, while the sample standard deviation is roughly 1.3763. There is also an acceptance rate of 0.6995 indicating a reasonable values of output.
2. *Figure 1.5* shows the convergence analysis ($\hat{R}$) for Metropolis-Hastings Algorithm over an s values between 0.001 and 1. When s_values is low, $\hat{R}$ is at its highest. However, as s gets closer to 1, $\hat{R}$ will start to decrease to a value of 1. Additionally, proper convergence is achieved when $\hat{R} < 1.05$.